

ADVANCED DATA MANAGEMENT 2023/2024
January 9, 2025
CLOSED BOOK

FIRST NAME:
SID:

LAST NAME:

Q1. (up to 10 lines) [**only for COMPUTER ENGINEERING STUDENTS**]
Discuss the leaderless replication protocol, explaining how read and write operations are performed and the conditions under which good properties for the read operation can be guaranteed.

Q2. (up to 10 lines)
Explain issues related to the execution of join operations in NoSQL systems: problems, solutions at the NoSQL system side and solutions at the application side.

Q3. (up to 10 lines)

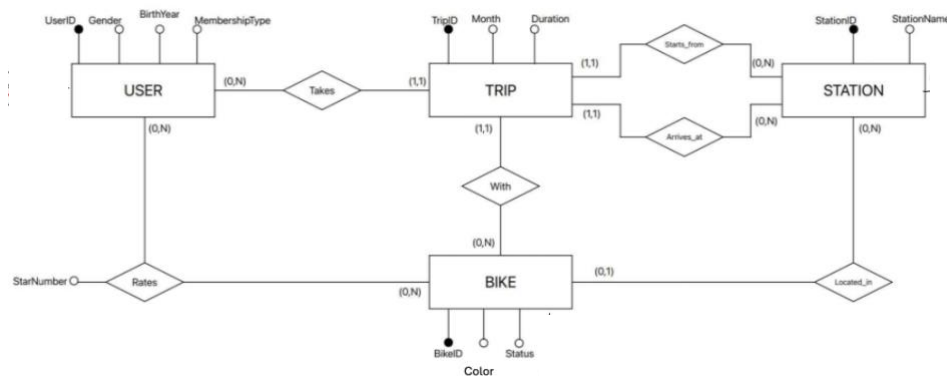
Discuss the pros and cons of graph-based systems, highlighting the most appropriate application contexts for their use and architectural issues.

Q4. (up to 10 lines) [**only for COMPUTER SCIENCE STUDENTS**]

Describe the main differences between RDF, RDFS, and OWL.

Exercise 1 [NoSQL aggregate-oriented data stores]

Consider the following entity relationship diagram:



The workload is the following:

- **Query 1:** Determine the id and the status of all the bikes.
- **Query 2:** Determine the id and the gender of the users born before 2000 that took a trip from station A10 to station A20 (A10 and A20 are station ids).
- **Query 3:** Determine the users that took a trip longer than 5 hours.

Based on the previous ER schema and workload:

1. Design, using the aggregate-oriented logical design methodology, the aggregates needed for answering the given workload queries. Present the output of each step of the methodology.
2. Assuming to represent the aggregates in MongoDB, choose a proper shard key and proper indexes for each of them. Motivate your answer and present the commands for creating indexes and shard keys in MongoDB for each aggregate.
3. Specify Query 2 in the MongoDB language. If your cluster contains 10 nodes, how many nodes will be used when executing Query 2?

1. Sketch of one possible solution

Formal representation of queries

Q1 (Bike, [], [Bike(BikeId, Status)_!])

Q2 (Trip , [User(Birthdate)_!, Station(StationID)_Sf , Station(StationId)_Aa], User(UserID, Gender)_!])

Notice that in this case TRIP is not a selection entity but, according to the schema, this choice simplifies the creation of the aggregate since TRIP belongs to the (1,1) part of the associations with STATION and USER. Other options are of course possible.

Q3 (Trip, [Trip(Duration)_!] , [User(UserId)_T])

Then, you should annotate the ER diagram starting from the queries.

Aggregates:

Bike (BikeId, Status)

Trip (TripId, Duration, UserId, Gender, Birthday, StationIdFrom, StationIdTo]

2.

Bike (associated queries: Q1)

Bike (BikeId, Status)

BikeId is the identifier, so we can create a unique index (in this case, the index is only used to maintain the uniqueness constraint on BikeId).

Since there is no selection condition, the query corresponds to a batch query, any partition key is fine. We choose _id (which is automatically added to the aggregate by MongoDB)

```
db.createCollection("Bike", {<schema>})
db.Bike.createIndex( { 'BikeId': 1 }, {unique: true})
db.adminCommand( { shardCollection: "db.Bike", key: { _id: 1 }, field: "hashed" } )
```

Trip (associated queries: Q2, Q3)

Trip (TripId, Duration, UserId, Gender, Birthday, StationIdFrom, StationIdTo]

The selection conditions refer to Duration (Q3) and (StationIdFrom, StationIdTo, Birthday) (Q2). In MongoDB we can create the partition key on one of these sets of selection attributes, e.g., on duration: in this case query Q2 will be executed using just one node. Then, we create a non-unique multi-attribute index on (StationIdFrom, StationIdTo, Birthday) to support query Q3.

```
db.createCollection("Trip", {<schema>})
db.adminCommand( { shardCollection: "db.Trip", key: { duration: 1 }, field: "hashed" } )
db.Trip.createIndex( { 'StationIdFrom': 1, 'StationIdTo': 1, 'Birthday': 1 }, {unique: false})
```

If we do not create any index on the partition key, a non-unique index will be automatically created.

As before and for the same motivations, a unique index can be created on TripId.

COMMENT

To better optimize queries Q2 and Q3, an alternative option is to create two different collections, one optimized for Q2 and one optimized for Q3 (as discussed for Cassandra). Any solution is however fine from the point of view of the written exam, provided that the queries can be executed over the aggregates, the partition keys and the indexes created.

3.

```
db.Trip.find( { "StationFromId" : "A10", "StationToID" : "A20", "Birthday" : { $lt : 2000 } }
```

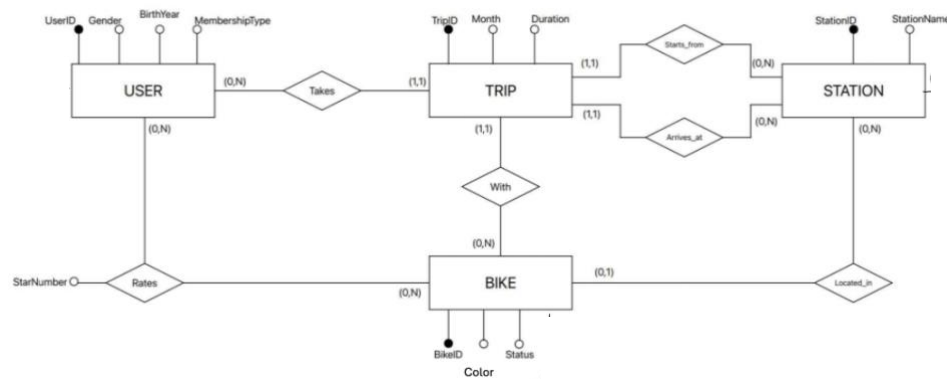
According to the selected partition key and the created indexes, since no equality condition for the partition key (duration) is provided, all the shards will be accessed. Inside each of them, the index will be used to detect the data to be returned. Thus, the number of accessed nodes correspond to the number of nodes in the cluster.

FURTHER COMMENT

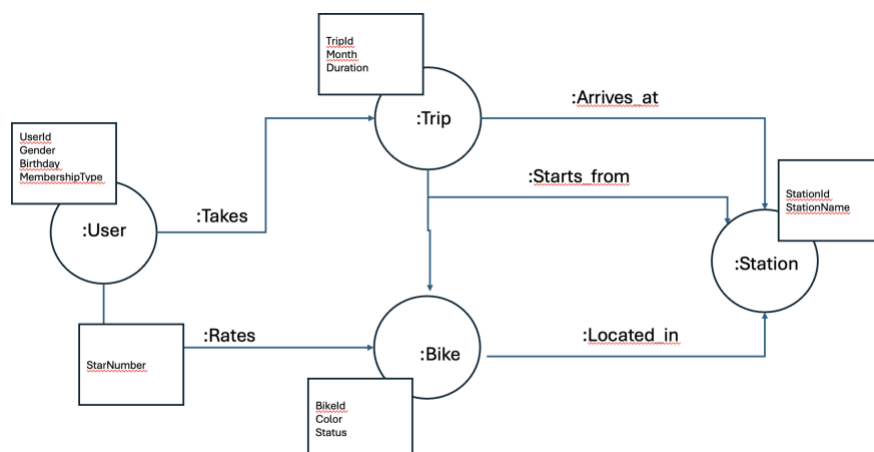
Here the important point is to provide an answer which is coherent with the solution proposed for item (2).

Exercise 2 [NoSQL graph-based data stores]

Consider the entity relationship diagram presented in Exercise 1:



1. Present the most suitable logical schema in a graph-based data store, like Neo4j (in the form of a graph pattern or of a sample instance), for the domain described by the



COMMENT

Notice that identifiers, like *BikeId*, should be included as properties in the schema only if they are a semantic meaning (surrogate identifiers are automatically made available for each node instance by the graph-based system).

2. Formulate in Cypher the query to determine the bikes that are currently located in a station which is not the destination of any trip in which the bike was used.

```
MATCH (b:Bike)-[:LOCATED_IN]->(s:Station)
WHERE NOT EXISTS {
    MATCH (b)-[:WITH]->(t:Trip)-[:TO]->(s)
}
RETURN b
```

Exercise 3 [Semantic data stores] [only for COMPUTER SCIENCE STUDENTS]

Consider the following extract from an RDF file:

```
ex:restaurant1 ex:hasName "Osteria Francescana" .
ex:restaurant1 ex:locatedIn ex:Modena .
ex:restaurant1 ex:hasChef ex:MassimoBottura .
ex:restaurant1 ex:awardedMichelinStars "3" .
ex:restaurant1 ex:servesCuisine ex:ItalianCuisine .

ex:restaurant2 ex:hasName "El Celler de Can Roca" .
ex:restaurant2 ex:locatedIn ex:Girona .
ex:restaurant2 ex:hasChef ex:JoanRoca .
ex:restaurant2 ex:awardedMichelinStars "3" .
ex:restaurant2 ex:servesCuisine ex:SpanishCuisine .

ex:MassimoBottura ex:hasNationality ex:Italian .
ex:JoanRoca ex:hasNationality ex:Spanish .
```

1. Add the class specifications for the classes **ex:Restaurant**, **ex:Cuisine**, and **ex:Chef**.

```
ex:Restaurant rdf:type rdfs:Class
ex:Cuisine rdf:type rdfs:Class
ex:Chef rdf:type rdfs:Class
```

2. Specify **ex:servesCuisine** as a property, and define its range and domain.

```
ex:servesCuisine rdf:type rdf:Property
ex:servesCuisine rdfs:domain ex:Restaurant
ex:servesCuisine rdfs:range ex:Cuisine
```

3. Specify **ex:StarredChef** as a subclass of **ex:Chef**. Is it possible and how to specify that a starred chef is a chef of a restaurant that is awarded Michelin Stars?

```
ex:StarredChef rdfs:subClassOf ex:Chef
```

It is not possible to specify that a starred chef is a chef of a restaurant awarded Michelin Stars with the constructs introduced in the course. In OWL, this can be specified using intensional classes (only cited in the course).

4. Is it possible and how to specify that a chef works in just one restaurant? Is it possible and how to specify that a restaurant has a single chef?

```
ex:hasChef rdf:type owl:inverseFunctionalProperty (a chef works in just one restaurant)
ex:hasChef rdf:type owl:functionalProperty (a restaurant has a single chef)
```

5. Write a SPARQL query to retrieve the names of all restaurants with 3 Michelin stars that serve Italian cuisine.

```
SELECT ?name WHERE
{
    ?r rdf:type ex:Restaurant.
    ?r ex:awardedMichelinStars "3".
    ?r ex:hasName ?name.
    ?r ex:servesCuisine ex:italianCuisine
}
```

6. Write a SPARQL query to retrieve the name, the chef, and (if available) the type of cuisine of all restaurants with Spanish chefs or located in Spain.

```
SELECT ?name, ?chef, ?type WHERE
{
    ?r rdf:type ex:Restaurant.
    ?r ex:hasName ?name.
    ?r ex:hasChef ?chef.
    OPTIONAL {?r ex:servesCuisine ?type}

    {?chef ex:hasNationality ex:Spanish}
    UNION
    {?r ex:locatedIn ex:Spain}
}
```

Hint (Cheat sheet RDF/S and OWL)

- instance relation `rdf:type` (classes are instances of `rdfs:Class`)
- subclass relation `rdfs:subClassOf`
- domain of a property `rdfs:domain`
- range of a property `rdfs:range`
- subproperty `rdfs:subPropertyOf`
- functional property `owl:functionalProperty`
- inverse functional property `owl:inverseFunctionalProperty`
- class disjointness `owl:disjointWith`
- instance equality and disequality `owl:sameAs` and `owl:differentFrom`